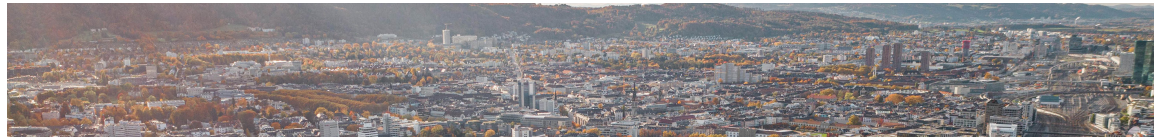




Universität
Zürich ^{UZH}

Department of Computational Linguistics



Naruto Hand Seal Detector

Advanced Machine Learning

Mert Erol, Ishana Rana, Chenxi Jiang, Ajeong Shin, Jenifer Meyer

May 30, 2026



Introduction

Background: Naruto Hand Seals

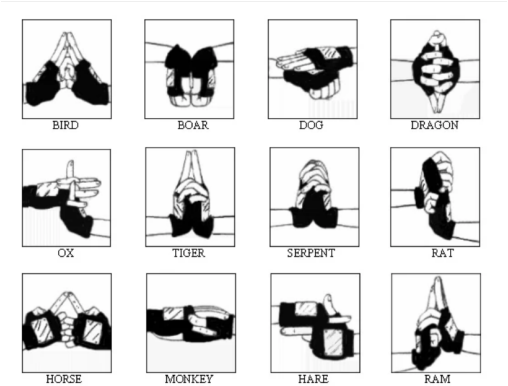
Naruto Hand Seals

- Used to perform many ninjutsu, genjutsu, and other secret arts
- 12 basic hand seals, each named after an animal of the Chinese zodiac
- In this project, we treat each seal as a visual gesture class to be recognized automatically

(Narutopedia contributors, n.d.)

Introduction

Background: Naruto Hand Seals



12 Basic Hand Seals (Shinobi Legends Wiki contributors, n.d.)

Introduction

Task Definition and Challenges

Task

- **Stage 1 — Seal classification:** given a live webcam frame, classify the performed hand seal into one of 12 classes in real time
- **Stage 2 — Jutsu recognition:** accumulate the predicted seal sequence within a time window and map known multi-seal combinations to a named jutsu (e.g. Ram → Snake → Tiger = Shadow Clone Jutsu)
- Predicted jutsu triggers a corresponding visual effect overlaid on the feed

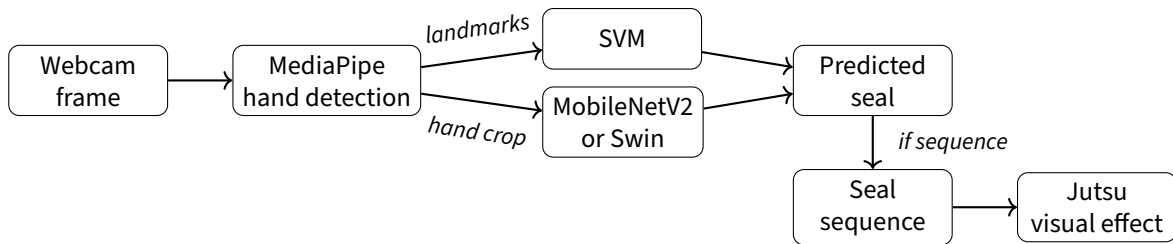
Key Challenges

- *Inter-class similarity* — subtle finger-position differences between seals (e.g. Tiger vs. Ram)
- *Limited & imbalanced data* — public dataset had 2,245 images with only a 3.8% test split
- *Real-time constraint* — model must be fast enough for smooth webcam inference

Introduction

Pipeline

Approach Overview



Introduction

Model Comparison

Three models compared

- **Baseline**

- *SVM*: kernel-based, uses MediaPipe landmark features

- **Advanced Models**

- *MobileNetV2*: lightweight CNN, transfer learning from ImageNet, optimised for real-time inference

- *Swin Transformer*: hierarchical Vision Transformer with shifted-window self-attention, strongest accuracy

All models were trained on the curated dataset described next.

Data

Dataset Collection

- Kaggle dataset by Vikranth Kanumuru (2020) → 2245 images
 - imbalanced class representation and low in diversity
 - too small test split (3.8%)

Kaggle Naruto Hand Seal Dataset Analysis

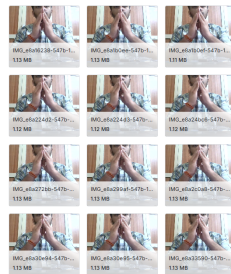
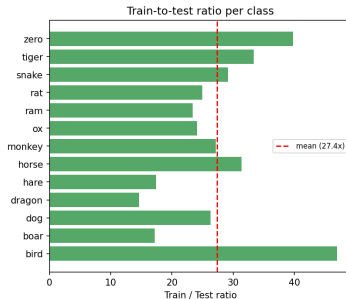
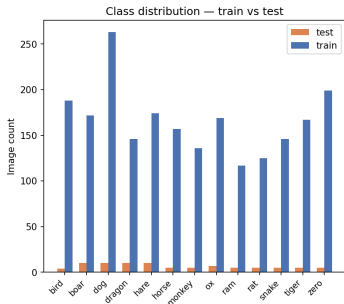


Figure: Screenshot
Dataset Bird Folder

Data

Adding to the Dataset

— Self-recorded clips

- Each team member recorded all 13 seals; transitions trimmed before extraction
- Varied subject distance (near vs. far) and slight angle variation per clip
- Frames extracted at 5 fps; stored under a consistent naming convention

— Roboflow dataset yylunxie (2026)

- 1200 images
- 80% training and 20% test

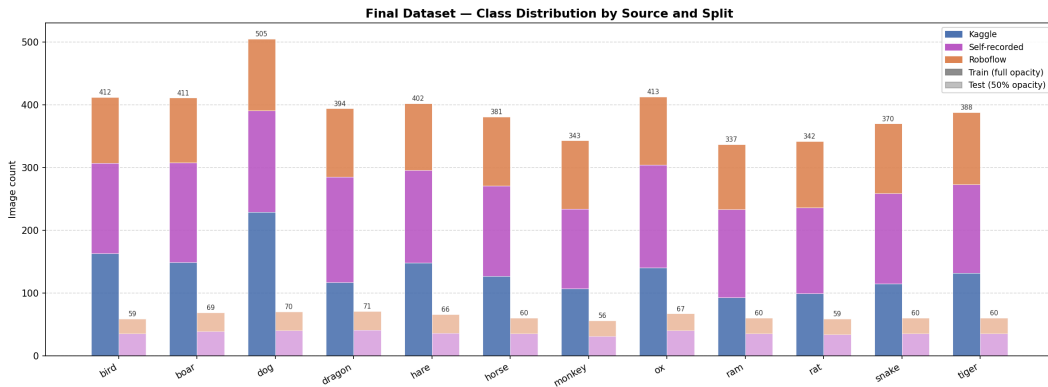


Figure: Screenshot of our own Dataset Bird Folder

Data

Dataset Curation

- Merged Kaggle (train + test), self-recorded, and Roboflow into a single pool → 5455 images
- Re-split into 80% train / 20% test



Method

SVM Model

Pro	Why it matters
Works in high dimensions	63–126 features, small sample size → SVM generalises well
Memory efficient	Only support vectors stored → fits embedded / real-time
Fast prediction	Dot product with few vectors → smooth 30 FPS inference
Non-linear boundaries	RBF kernel captures complex finger position relationships
Deterministic	No randomness after training → consistent results

Method

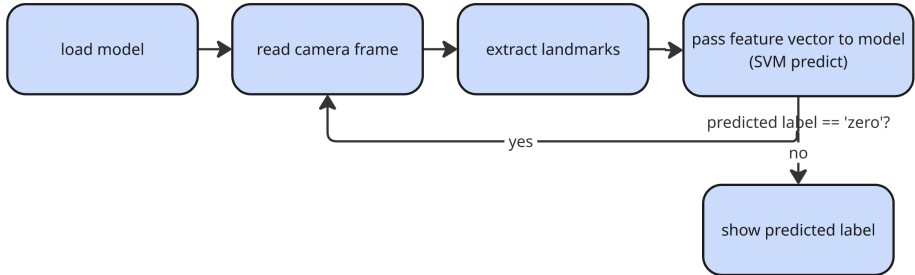
SVM Model

Con	Implication
Poor scaling to large data	Training becomes slow if dataset > 50k images
Requires feature scaling	Landmarks not normalised in current code → may hurt accuracy
Default hyperparameters	C and gamma not tuned – suboptimal
Sensitive to class imbalance	No class weights → rare gestures may be ignored
Not interpretable	Hard to explain why a specific hand pose was predicted
Probability calibration overhead	Platt scaling adds training time, may be unreliable

Method

SVM Model

during inference:



Method

SVM Model

issues:

- no feature normalization
- SVM does not scale to large datasets
- untuned C & γ (default values) may lead to under- or overfitting

Method

MobileNetV2 — Architecture

What is MobileNetV2?

A lightweight CNN pretrained on ImageNet, designed for efficient image classification.

Key building blocks:

- **Depthwise separable convolutions:** drastically fewer parameters than standard CNNs
- **Pretrained:** transfers general visual features learned from 1M+ images

Our modifications:

Replaced the original classifier head with:

1. Dropout ($p = 0.6$)
2. Linear layer $\rightarrow$ 256 units
3. ReLU activation
4. Dropout ($p = 0.5$)
5. Linear layer $\rightarrow$ 12 classes

Strong dropout chosen to prevent overfitting given the limited dataset size.

Method

MobileNetV2 — Training Strategy

Two-phase fine-tuning:

Phase 1 — Head only (15 epochs)

- Backbone fully frozen
- Only classification head trained
- LR = $5e-4$
- Lets the head adapt before touching pretrained weights

Phase 2 — Fine-tuning (up to 30 epochs)

- Last 7 backbone blocks unfrozen
- Backbone LR = $1e-4$ ($10\times$ lower)
- Early stopping with patience = 8
- OneCycleLR scheduler with 30% warmup

Method

MobileNetV2 — The Cropping Problem

Without hand cropping:

- Model sees full frame: background, clothing, face
- Learns spurious features instead of hand shape
- Test accuracy: **62.9%**
- Bird and Ram heavily confused (11 misclassifications)

With MediaPipe hand cropping:

- Tight crop around hand region only
- Forces model to focus on finger details
- Test accuracy: **98.4%**
- Same confusions reduced to 1–2 isolated errors

Hand cropping was the single most impactful change
→ **+35.5%** accuracy gain with no architecture changes

Method

Swin Transformer — Architecture

What is Swin Transformer?

A pure Vision Transformer — no convolutions at all. Processes images as sequences of patches with self-attention.

Key idea:

- Standard transformers compute **global attention** — every patch attends to every other patch $\rightarrow O(N^2)$
- Swin uses **Local shifted windows:** attention computed within small regions

Our modifications:

Replaced the original classification head with:

1. LayerNorm
2. Dropout ($p = 0.4$)
3. Linear layer $\rightarrow$ 256 units
4. GELU activation
5. Dropout ($p = 0.3$)
6. Linear layer $\rightarrow$ 12 classes

GELU used instead of ReLU: smoother gradient flow.

Method

Swin Transformer — Training Strategy

Phase 1 — Head only (10 epochs)

- Backbone fully frozen
- Only classification head trained
- $LR = 3e-4$
- Val accuracy after Phase 1: **79.9%**
- MobileNetV2 reached only 63.5% at same stage

Phase 2 — Fine-tuning (up to 30 epochs)

- Last 4 Swin stages unfrozen
- Backbone $LR = 5e-5$ ($6\times$ lower)
- Early stopping with patience = 8
- OneCycleLR scheduler with 30% warmup

Why does Phase 1 start stronger? Swin's attention-based pretrained features transfer better to hand gesture classification — the model already understands spatial relationships between image regions out of the box.

Method

Swin Transformer — Interpretability with EigenCAM

What does the model actually look at?

Used **EigenCAM** to visualize which regions of each image drive the model's predictions.

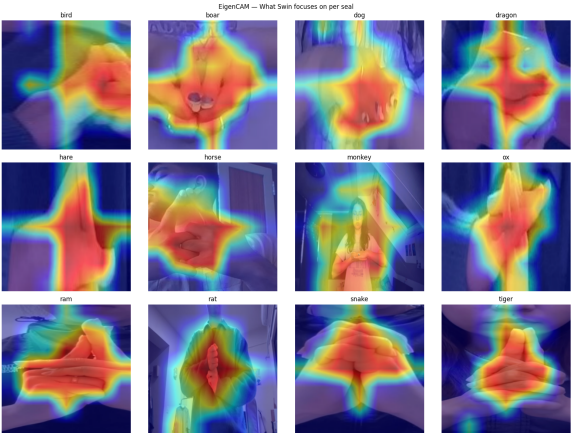
- Decomposes the last Swin stage's feature map
- Highlights spatially important regions

What we found:

- Model consistently focuses on **finger and knuckle regions**
- Background, face, and clothing are largely ignored
- Model learned the right features
- Provides interpretability that MobileNetV2 cannot offer without additional tooling

Method

Swin Transformer - Interpretability with EigenCam



Evaluation

— Metrics

- **Top-1 Accuracy:** Primary metric for the balanced 12-class classification task.
- **Top-3 Accuracy:** Checks whether the correct class is ranked highly, useful for visually similar seals.
- **Confusion Matrix:** Reveals systematic confusion between specific classes.

— Validation Strategy

- **5-Fold Cross-Validation:** Provides more stable estimates during model selection and hyperparameter tuning.

Evaluation

SVM Model - Model Metrics

- Top-1 Accuracy: 0.501
- Top-3 Accuracy: 0.821

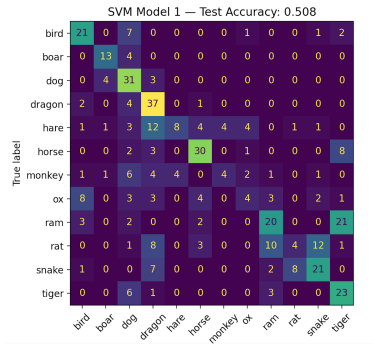


Figure: MobileNetV2 Confusion Matrix

Evaluation

SVM Model - Model Stability

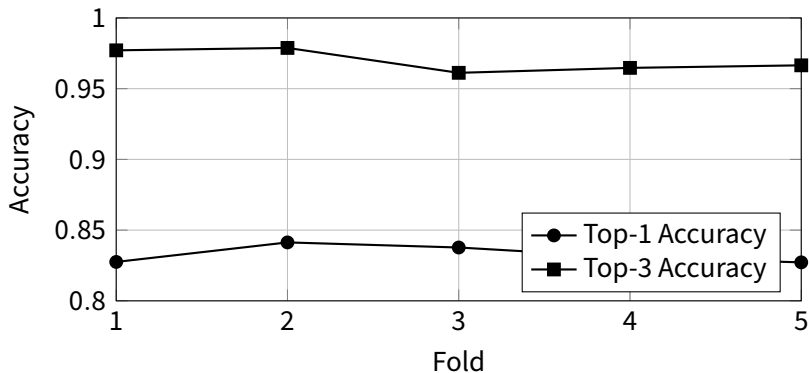


Figure: Top-1 and Top-3 accuracy of the SVM baseline across five folds.

Evaluation

MobileNetV2 Model - Metrics

- top1-accuracy: 0.981
- top3-accuracy: 0.999

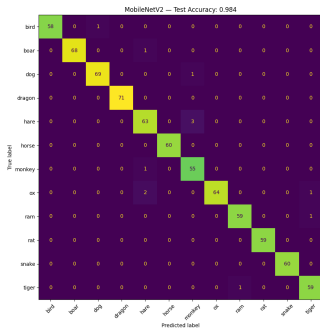


Figure: MobileNetV2 Confusion Matrix

Evaluation

MobileNetV2 Model - Model Stability

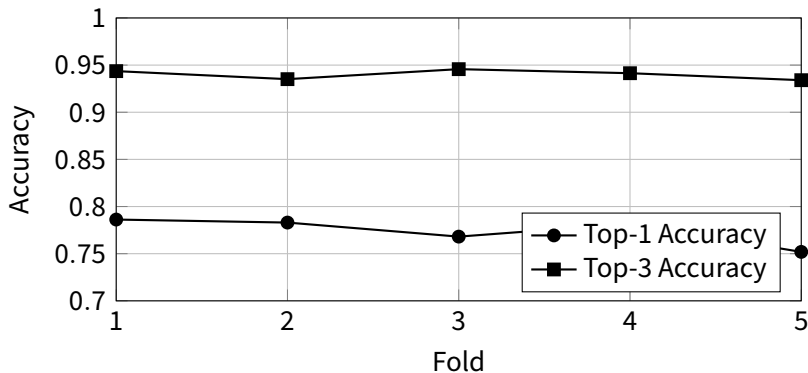


Figure: Top-1 and Top-3 accuracy of MobileNetV2 across five folds.

Evaluation

MobileNetV2 Model - Model Stability

- MobileNetV2 model has a accuracy around 0.77.
- MobileNetV2 model has a low standard deviation which means its performance was consistent across different validation splits.

Metric	Mean	Std
Top-1 Accuracy	0.7742	0.0127
Top-3 Accuracy	0.9400	0.0047

Table: 5-fold cross-validation stability result of MobileNetV2.

Evaluation

Swin Model - Metrics

- top1-accuracy: 0.996
- top3-accuracy: 0.999

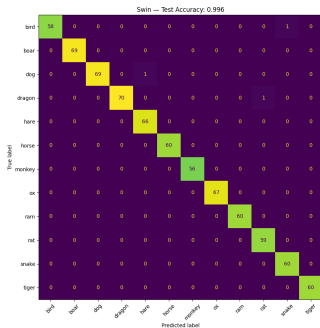


Figure: Swin Confusion Matrix

Evaluation

Swin Model - Model Stability

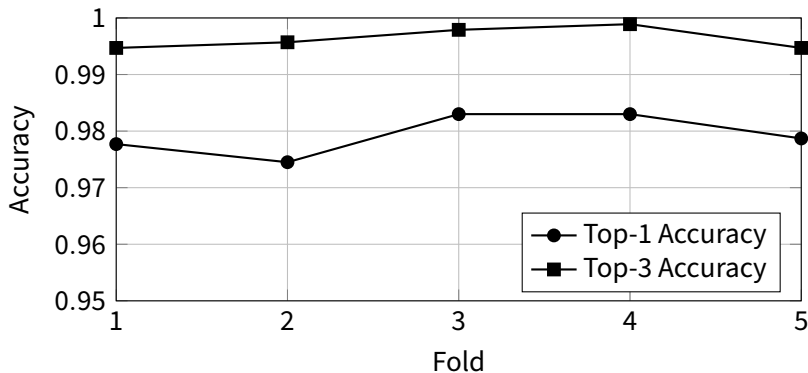


Figure: Top-1 and Top-3 accuracy of Swin-S across five folds.

Evaluation

Swin Model - Model Stability

- Swin model has a very high accuracy.
- Swin model has a very low standard deviation which means its performance was very consistent across different validation splits.

Metric	Mean	Std
Top-1 Accuracy	0.9794	0.0033
Top-3 Accuracy	0.9964	0.0017

Table: 5-fold cross-validation stability result of Swin-S.

Evaluation

Accuracy Comparison

Model	Test Accuracy
SVM	0.501
MobileNetV2	0.984
Swin-S	0.996
Swin-S + TTA	0.996

Conclusion

<i>Model</i>	SVM	MobileNetV2	Swin Transformer
<i>Architecture Type</i>	Traditional (Kernel-based)	Lightweight CNN	Pure Vision Transformer
<i>Input Strategy</i>	Raw Landmarks	MediaPipe Crops	Local Attention
<i>Key Advantage</i>	Memory Efficient	Inference Speed	Interpretability
<i>Accuracy</i>	50.1%	98.4%	99.6%

Live Demo

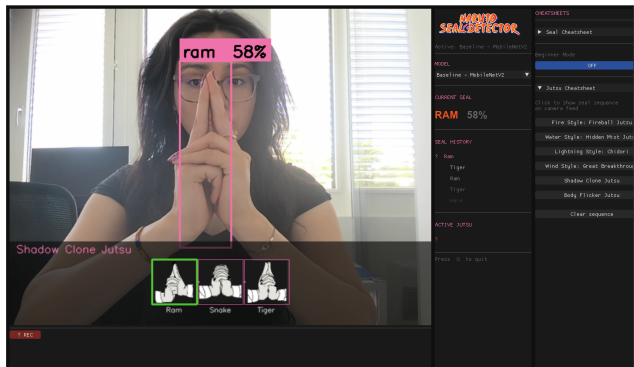
Github Repo Group 18

How to Run the Application

1. Activate the virtual environment
2. Navigate to the app folder
3. Run: `python main.py`

Frontend built with:

- OpenCV
- DearPyGUI



Thank you for your attention!



References I

Vikranth Kanumuru. 2020. Naruto hand sign dataset.

<https://www.kaggle.com/datasets/vikranthkanumuru/naruto-hand-sign-dataset>.
Kaggle dataset.

Narutopedia contributors. n.d. Hand seal. https://naruto.fandom.com/wiki/Hand_Seal.
Accessed: 2026-04-30.

Shinobi Legends Wiki contributors. n.d. Hand seals.

https://shinobilegendsprofile.fandom.com/wiki/Hand_Seals. Accessed:
2026-05-01.

yylunxie. 2026. Naruto hand sign dataset.

<https://universe.roboflow.com/yylunxie/naruto-hand-sign-p8toe/dataset/1>.
Version 1; accessed: 2026-05-05.